



C++ - 모듈 04

하위 유형 다형성, 추상 클래스, 인터페이스

요약:

이 문서에는 C++ 모듈의 모듈 04 연습이 포함되어 있습니다.

버전: 11

내용물

나	소개	2
II	일반 규칙	삼
III	연습 00: 다형성	5
IV	연습 01: 나는 세상에 불을 지르고 싶지 않다	7
V	연습 02: 추상 클래스	9
VI	연습 03: 인터페이스 및 요약	10
VII	제출 및 동료 평가	14

제1장

소개

C++는 Bjarne Stroustrup이 C 프로그래밍 언어 또는 "C with Classes"의 확장으로 만든 범용 프로그래밍 언어입니다(출처: [위키피디아](#)).

이 모듈의 목표는 다음을 소개하는 것입니다. **객체 지향 프로그래밍**. 이것이 C++ 여정의 출발점이 될 것입니다. OOP를 배우려면 많은 언어를 권장합니다. C++는 오랜 친구인 C에서 파생되었기 때문에 선택하기로 결정했습니다. C++는 복잡한 언어이고 단순함을 유지하기 위해 코드는 C++98 표준을 준수합니다.

우리는 현대 C++가 여러 측면에서 많이 다르다는 것을 알고 있습니다. 따라서 능숙한 C++ 개발자가 되고 싶다면 42 Common Core 이후에 더 나아가는 것은 여러분에게 달려 있습니다!

제2장

일반 규칙

컴파일 중

- 다음을 사용하여 코드를 컴파일하세요. `g++` 그리고 깃발 `-Wextra -Werror`
- 플래그를 추가해도 코드는 계속 컴파일되어야 합니다. 표준 = C++98

형식 및 명명 규칙

- 연습 디렉토리의 이름은 다음과 같이 지정됩니다. `ex00`, `ex01`, ... , 엑스
- 지침에서 요구하는 대로 파일, 클래스, 함수, 멤버 함수 및 속성의 이름을 지정하십시오.
- 클래스 이름을 적습니다. **어퍼카멜케이스** 체제. 클래스 코드가 포함된 파일의 이름은 항상 클래스 이름에 따라 지정됩니다. 예를 들어:
클래스 이름. `hpp`/클래스 이름. `h`, 클래스 이름. `cpp`, 또는 클래스명. `tpp`. 그런 다음 벽돌 벽을 나타내는 "BrickWall" 클래스 정의가 포함된 헤더 파일이 있는 경우 해당 이름은 다음과 같습니다. `BrickWall.hpp`.
- 달리 지정하지 않는 한, 모든 출력 메시지는 개행 문자로 끝나야 하며 표준 출력에 표시되어야 합니다.
- *안녕 노르미네!* C++ 모듈에는 코딩 스타일이 적용되지 않습니다. 당신이 좋아하는 것을 따라갈 수 있습니다. 하지만 동료 평가자가 이해할 수 없는 코드는 채점할 수 없는 코드라는 점을 명심하세요. 깨끗하고 읽기 쉬운 코드를 작성하기 위해 최선을 다하십시오.

허용/금지

더 이상 C로 코딩하지 않습니다. C++를 배울 시간입니다! 그러므로:

- 표준 라이브러리의 거의 모든 것을 사용할 수 있습니다. 따라서 이미 알고 있는 내용을 고수하는 대신 익숙한 C 함수의 C++ 버전을 최대한 많이 사용하는 것이 현명할 것입니다.
- 그러나 다른 외부 라이브러리는 사용할 수 없습니다. 이는 C++11(및 파생 형식)을 의미하며 후원도서관은 금지되어 있습니다. 다음 기능도 금지됩니다. `*printf()`, `*alloc()` 그리고 무로(`).` 사용하면 성적이 0이 되고 그게 전부입니다.

- 별도로 명시하지 않는 한, 네임스페이스 <ns_name> 사용 그리고 친구 키워드는 금지되어 있습니다. 그렇지 않으면 성적은 -42가 됩니다.
- **모듈 08과 09에서만 STL을 사용할 수 있습니다.** 즉, **아니오 컨테이너**(벡터/목록/맵 등) 및 **아니오 알고리즘**(<를 포함해야 하는 모든 것 알고리즘> 헤더) 그때까지. 그렇지 않으면 성적은 -42가 됩니다.

몇 가지 디자인 요구 사항

- C++에서도 메모리 누수가 발생합니다. 메모리를 할당할 때(새로운 키워드)를 피해야 합니다. **메모리 누수.**
- 모듈 02부터 모듈 09까지 수업은 다음과 같이 설계되어야 합니다. **달리 명시적으로 언급된 경우를 제외하고 정교회 정경 형식.**
- 헤더 파일에 있는 모든 함수 구현(함수 템플릿 제외)은 실습에서 0을 의미합니다.
- 각 헤더를 다른 헤더와 독립적으로 사용할 수 있어야 합니다. 따라서 필요한 모든 종속성을 포함해야 합니다. 그러나 다음을 추가하여 이중 포함 문제를 피해야 합니다. **경비원을 포함하다.** 그렇지 않으면 성적은 0이 됩니다.

나를 읽어주세요

- 필요한 경우(예: 코드 분할) 몇 가지 추가 파일을 추가할 수 있습니다. 이러한 과제는 프로그램에 의해 확인되지 않으므로 필수 파일을 제출하는 한 자유롭게 확인하십시오.
- 때로는 연습 지침이 짧아 보이지만 지침에 명시적으로 기록되지 않은 요구 사항이 예제를 통해 표시될 수 있습니다.
- 시작하기 전에 각 모듈을 완전히 읽으십시오! 정말로, 그렇게 하세요.
- 오딘이, 토르가! 머리를 써라!!!



많은 클래스를 구현해야 합니다. 즐겨 사용하는 텍스트 편집기를 스크립트로 작성할 수 없다면 이는 지루해 보일 수 있습니다.



연습을 완료할 수 있는 어느 정도의 자유가 주어집니다. 그러나 필수 규칙을 따르고 게으르지 마십시오. 당신은 많은 유용한 정보를 놓칠 것입니다! 주저하지 말고 이론적 개념에 대해 읽어보세요.

제3장

연습 00: 다형성

	운동 : 00
다형성	
제출 디렉토리: 전-00/	
제출할 파일:메이크파일, main.cpp, *.cpp, *.{h, hpp}	
금지된 기능:없음	

모든 운동에 대해 다음을 제공해야 합니다.**가장 완벽한 테스트**당신은 할 수 있습니다. 각 클래스의 생성자와 소멸자는 특정 메시지를 표시해야 합니다. 모든 클래스에 동일한 메시지를 사용하지 마세요.

다음과 같은 간단한 기본 클래스를 구현하여 시작합니다.**동물**. 여기에는 하나의 보호된 속성이 있습니다.

- 표준::문자열 유형;

구현**개**Animal을 상속받은 클래스입니다. 구현**고양이**Animal을 상속받은 클래스입니다.

이 두 파생 클래스는 해당 클래스를 설정해야 합니다.유형이름에 따라 필드를 지정합니다. 그러면 Dog의 타입은 "Dog"로 초기화되고, Cat의 타입은 "Cat"으로 초기화됩니다. Animal 클래스의 유형은 비워두거나 원하는 값으로 설정할 수 있습니다.

모든 동물은 멤버 함수를 사용할 수 있어야 합니다. 메이크사운드()
적절한 소리를 인쇄합니다(고양이는 짹지 않습니다).

이 코드를 실행하면 Animal 클래스가 아닌 Dog 및 Cat 클래스의 특정 사운드가 인쇄됩니다.

```
정수기본()
{
    const동물* 메타 =새로운동물(); const동물* j
    =새로운개(); const동물* 나 =새로운고양이();

    std::cout << j->getType() << " "<< 표준::endl; std::cout <<
    i->getType() << " "<< 표준::endl; i->makeSound(); //고양이/
    소리를 출력합니다! j->makeSound();

    메타->makeSound();
    ...

    반환0;
}
```

작동 방식을 이해하려면 다음을 구현하세요. **틀린고양이** 클래스를 상속받은 클래스 **잘못된동물** 수업. 위 코드에서 Animal과 Cat을 잘못된 것으로 바꾸면 WrongCat은 WrongAnimal 사운드를 출력해야 합니다.

위에 제공된 것보다 더 많은 테스트를 구현하고 제출하십시오.

제4장

연습 01: 나는 세상에 불을 지르고 싶지 않다

	운동 : 01
난 세상에 불을 지르고 싶지 않아	
제출 디렉토리: 전-01/	
제출할 파일: 이전 연습의 파일 + *.cpp, *.{h, hpp}	
금지된 기능: 없음	

각 클래스의 생성자와 소멸자는 특정 메시지를 표시해야 합니다.

구현 뇌수업. 100개의 배열이 포함되어 있습니다. 표준::문자열~라고 불리는 아이디어. 이렇게 하면 Dog와 Cat은 개인 정보를 갖게 됩니다. 뇌*기인하다.
건설 시 개와 고양이는 다음을 사용하여 두뇌를 생성합니다. 새로운 뇌(); 파괴되면 개와 고양이는 삭제그들의 두뇌.

기본 함수에서 다음 배열을 만들고 채웁니다. 동물사물. 절반은 그럴거예요 개객체가 되고 나머지 절반은 고양이사물. 프로그램 실행이 끝나면 이 배열을 반복하고 모든 Animal을 삭제합니다. 개와 고양이를 동물로 직접 삭제해야 합니다. 적절한 소멸자는 예상 순서로 호출되어야 합니다.

확인하는 것을 잊지 마세요 메모리 누수.

개나 고양이의 사본은 알아서는 안 됩니다. 따라서 복사본이 전체 복사본인지 테스트해야 합니다!


```
정수기본()
{
    const동물*j=새로운개(); const동물
    *나=새로운고양이();

    삭제제이;//누출이 발생하면 안 됩니다. 삭제나;

    ...

    반환0;
}
```

위에 제공된 것보다 더 많은 테스트를 구현하고 제출하십시오.

제5장

연습 02: 추상 클래스

	운동 : 02
추상 수업	
제출 디렉토리: 전-02/	
제출할 파일:이전 연습의 파일 + *.cpp, *.{h, hpp}	
금지된 기능:없음	

결국 동물 개체를 만드는 것은 의미가 없습니다. 사실이에요. 소리가 나지 않거든요!

가능한 실수를 방지하려면 기본 Animal 클래스를 인스턴스화해서는 안 됩니다. 누구도 인스턴스화할 수 없도록 Animal 클래스를 수정하세요. 모든 것이 이전처럼 작동해야 합니다.

원하는 경우 Animal에 A 접두사를 추가하여 클래스 이름을 업데이트할 수 있습니다.

제6장

연습 03: 인터페이스 및 요약

	운동 : 03
	인터페이스 및 요약
	제출 디렉토리: 전-03/
	제출할 파일:메이크파일, main.cpp, *.cpp, *.{h, hpp}
	금지된 기능:없음

C++98에는 인터페이스가 없습니다(C++20에도 없음). 그러나 순수 추상 클래스를 일반적으로 인터페이스라고 합니다. 따라서 이 마지막 연습에서는 이 모듈이 있는지 확인하기 위해 인터페이스를 구현해 보겠습니다.

다음의 정의를 완성하세요.**AMateria**클래스를 만들고 필요한 멤버 함수를 구현합니다.

```
클래스 A재료
{
    보호됨:
        [...]

    공공의:
        AMateria(std::string const & type); [...]

        std::string const & getType() const; //물질 유형을 반환합니다.

        가상 AMateria* clone() const = 0; 가상 무효 사
        용(ICharacter& 대상);
};
```

Materia 구체적인 클래스 구현 **얼음** 그리고 **치유법**. 유형을 설정하려면 이름을 소문자(Ice의 경우 "ice", Cure의 경우 "cure")를 사용하십시오. 물론, 그들의 멤버 기능은 클론() 동일한 유형의 새 인스턴스를 반환합니다(즉, Ice Materia를 복제하면 새 Ice Materia를 얻게 됩니다).

그만큼사용(ICharacter&)멤버 함수가 표시됩니다:

- 얼음: `"*<name>에게 아이스볼트를 발사합니다 *"`
- 치유법: `"*<name>의 상처를 치료해 줍니다 *"`

<이름>매개변수로 전달된 캐릭터의 이름입니다. 꺾쇠 괄호(< 및 >)를 인쇄하지 마세요.



Materia를 다른 Materia에 할당하는 동안 유형을 복사하는 것은 의미가 없습니다.

구체적인 클래스 작성 **성격** 다음 인터페이스를 구현합니다.

```
클래스 ICharacter
{
    공공의:
        가상 ~ICharacter() {}
        가상 std::string const & getName() const = 0; 가상 보이드
        장비(AMateria* m) = 0;
        가상 무효 unequip(int idx) = 0;
        가상 무효 사용(int idx, ICharacter& 대상) = 0;
};
```

그만큼**성격** 4개 슬롯의 인벤토리를 보유하고 있으며 이는 최대 4개의 마테리아를 의미합니다. 건설 중에는 재고가 비어 있습니다. 그들은 발견한 첫 번째 빈 슬롯에 마테리아를 장착합니다. 이는 슬롯 0에서 슬롯 3까지의 순서를 의미합니다. 전체 인벤토리에 마테리아를 추가하려고 하거나 존재하지 않는 마테리아를 사용/장착 해제하려는 경우에는 아무 것도 하지 마십시오(그래도 버그는 금지되어 있습니다). 그만큼장비 해제()멤버 함수는 Materia를 삭제하면 안 됩니다!



캐릭터가 바닥에 남겨둔 마테리아를 원하는 대로 처리하세요. unequip()이나 다른 것을 호출하기 전에 주소를 저장하십시오. 그러나 메모리 누수를 피해야 한다는 것을 잊지 마십시오.

그만큼사용(int, ICharacter&)멤버 함수는 슬롯에서 Materia를 사용해야 합니다.idx],대상 매개변수를AMateria::사용기능.



캐릭터의 인벤토리는 모든 유형의 AMateria를 지원할 수 있습니다.

당신의 **성격** 이름을 매개변수로 사용하는 생성자가 있어야 합니다. 캐릭터의 모든 복사본(복사 생성자 또는 복사 할당 연산자 사용)은 다음과 같아야 합니다. **깊은** 복사하는 동안 캐릭터의 마테리아를 삭제해야 새 마테리아가 인벤토리에 추가됩니다. 물론, 캐릭터가 파괴되면 마테리아도 삭제되어야 합니다.

구체적인 클래스 작성 **물질소스** 다음 인터페이스를 구현합니다.

클래스 IMateriaSource

```
{
    공공의:
        가상 ~IMateriaSource() {}
        가상 무효 learnMateria(AMateria*) = 0;
        virtual AMateria* createMateria(std::string const & type) = 0;
};
```

- learnMateria(AMateria*)
매개변수로 전달된 Materia를 복사하여 나중에 복제할 수 있도록 메모리에 저장합니다. 캐릭터와 마찬가지로, **물질소스** 최대 4개의 물질을 알 수 있습니다. 반드시 고유한 것은 아닙니다.
- createMateria(std::string const &)
새로운 마테리아를 반환합니다. 후자는 이전에 배운 Materia의 복사본입니다. **물질소스** 해당 유형은 매개변수로 전달된 유형과 같습니다. 유형을 알 수 없는 경우 0을 반환합니다.

간단히 말해서, 당신의 **물질소스** 필요할 때 이를 생성하려면 물질의 "템플릿"을 배울 수 있어야 합니다. 그런 다음 해당 유형을 식별하는 문자열만 사용하여 새 Materia를 생성할 수 있습니다.

이 코드를 실행하면:

```
정수기본()
{
    IMateriaSource* src = 새로운물질소스(); src->learnMateria(새로운얼음()); src->learnMateria(새로운치유법());

    ICharacter* 나 = 새로운성격("나");

    AMateria* tmp;
    tmp = src->createMateria("얼음"); 나->장비(tmp);
    tmp = src->createMateria("치유법"); 나->장비(tmp);

    ICharacter* 밥 = 새로운성격("단발");

    나->사용(0, *밥); 나->사용(1, *밥);

    삭제단발;
    삭제나;
    삭제src;

    반환0;
}
```

다음을 출력해야 합니다:

```
$> clang++ -W -Wall -Werror *.cpp $> ./
a.out | 고양이 -e
* 밥에게 아이스볼트를 쏜다 *$
* 밥의 상처를 치료함 *$
```

평소와 같이 위에 제공된 것보다 더 많은 테스트를 구현하고 제출하십시오.



연습 03을 수행하지 않고도 이 모듈을 통과할 수 있습니다.

제7장

제출 및 동료 평가

과제를 제출하세요. 힘내평소대로 저장소. 방어 중에는 저장소 내부 작업만 평가됩니다. 주저하지 말고 폴더와 파일의 이름이 올바른지 다시 확인하세요.



????????????? XXXXXXXXXXXX = \$3\$6b616b91536363971573e58914295d42